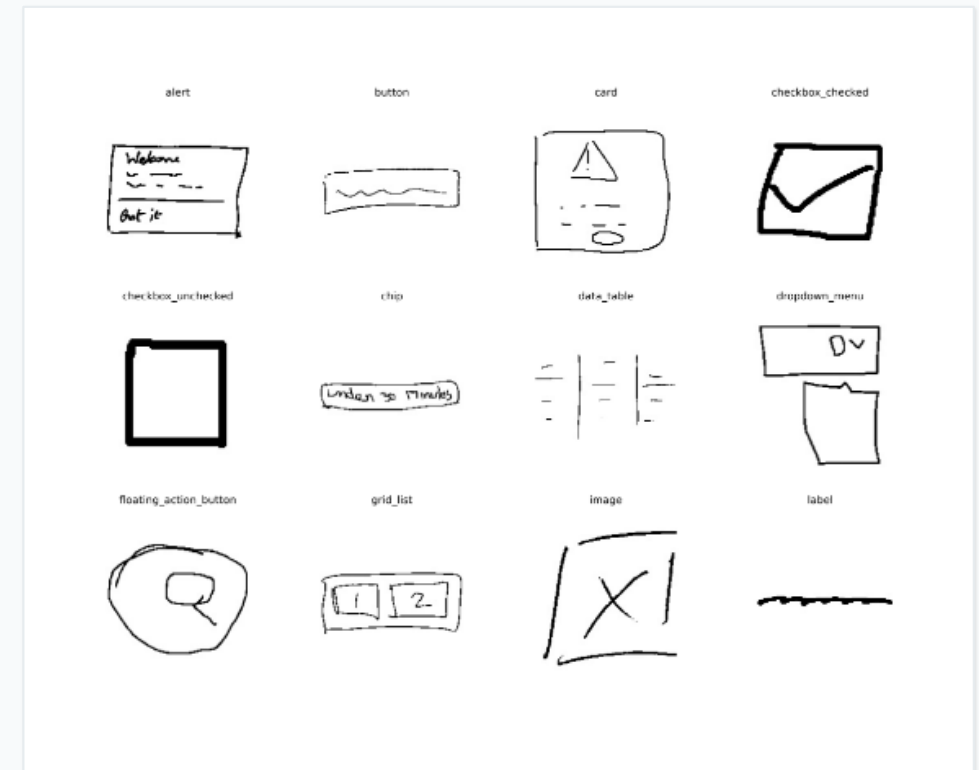


UI Sketch Element Classification

From hand-drawn UI sketches to predicted interface components

Image Processing and Computer Vision · SoSe 2026
Prof. Temerinac-Ott

Anjali Jitendrabhai Barvaliya · Matrikel-Nr. 287009
Chitroda Vishal · Matrikel-Nr. 287182



UISketch sample elements

19,898
images

21
UI classes

Why UI sketch classification matters



UI/UX work often begins as quick sketches. Computer vision can make those early design artifacts searchable, measurable, and eventually executable.

Design origin

Rough sketches and wireframes are fast, expressive, and intentionally incomplete.

AI opportunity

A classifier turns drawings into structured UI element labels that downstream tools can use.

Future relevance

The same capability can support sketch-to-interface and design-to-code workflows.

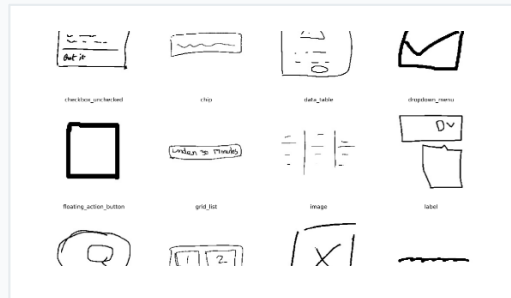
Project relevance

This project bridges design thinking with applied AI: it tests whether CNN-based models can recognize visual patterns in human-made interface sketches.

Supervised multi-class classification of UI sketch elements

Research question

How effectively can CNN-based models recognize visually variable hand-drawn UI components?



Input: one sketch image

224 × 224 pixels



CNN-based model
learned visual features



Output: one UI class
among 21 categories

- Examples: button, text field, checkbox, icon, dropdown, slider
- Core challenge: sketches vary by person and several UI elements are visually similar

UISketch is large enough for proof-of-concept, but still class-sensitive

19,898

images

21

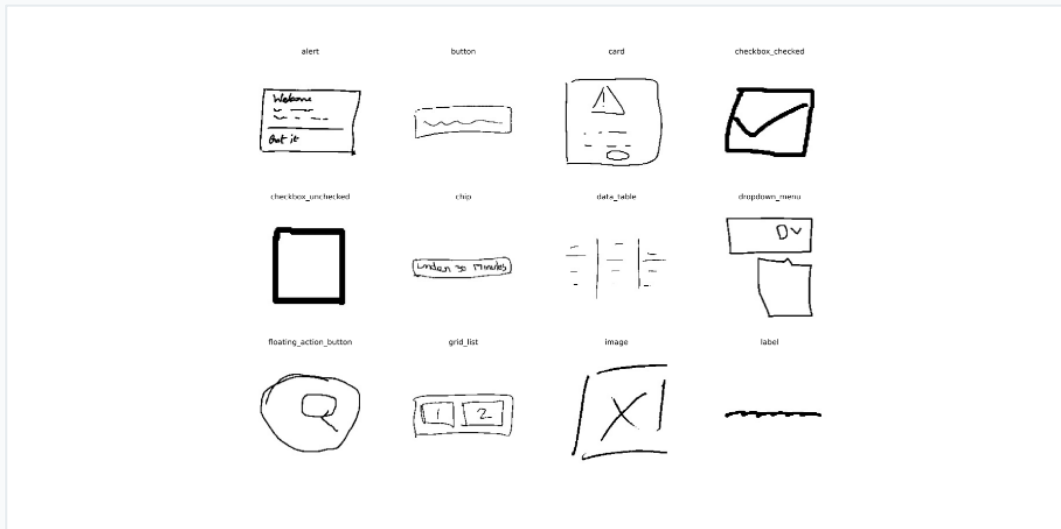
classes

224×224

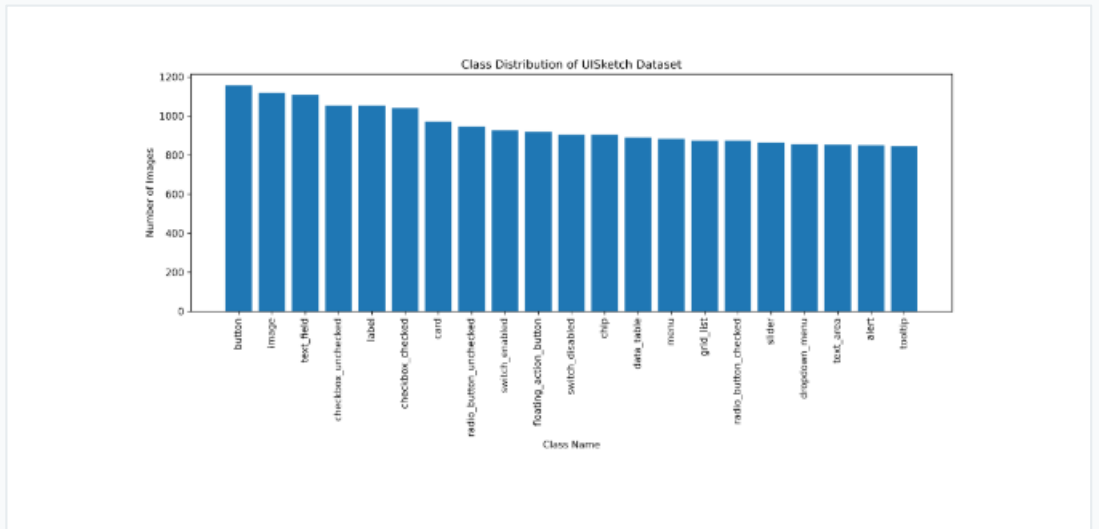
input size

32

batch size



Sample UI element sketches: button, checkbox, text field, text area, image, card, dropdown menu, slider, switch, tooltip, label, icon



Class distribution is not identical, making macro-level metrics important.

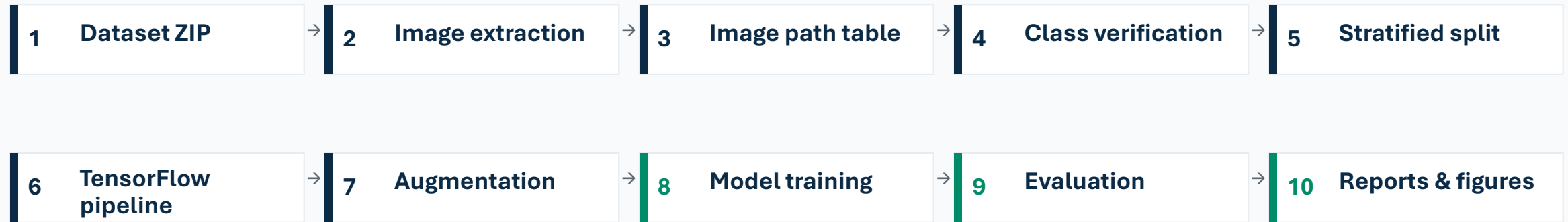
Stratified split



Train 13,928 · Validation 2,985 · Test 2,985

From Kaggle ZIP to auditable model reports

The workflow was designed for comparability: every model uses the same data split, input size, evaluation logic, and reporting structure.



Reproducibility safeguards

- Fixed random seed and saved split files
- Training logs captured per model
- Classification reports, confusion matrices, and misclassified examples produced for diagnosis

One fair protocol for all three models

All experiments share the same train/validation/test split, 224×224 image input, and evaluation logic so that model differences reflect model strategy rather than pipeline drift.

Data split



Evaluation evidence

Accuracy

overall correctness

Recall

class coverage

Confusion matrix

where classes mix

Precision

prediction reliability

Macro F1

balanced class-wise score

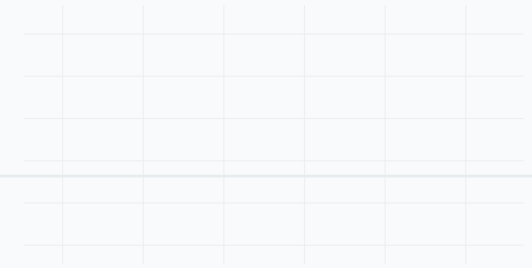
Misclassified examples

qualitative diagnosis

Why macro F1 is the decision metric

Macro F1 weights every class equally. This matters when similar or less frequent UI classes should not be hidden by high performance on easier, more common classes.

Model maturity: baseline → transfer learning → fine-tuning



1 Custom CNN

Simple baseline trained from scratch to establish a credible reference point.



2 EfficientNetB0 frozen

Transfer learning with pretrained visual features; head learns UI classes.



3 EfficientNetB0 fine-tuned

Deeper layers adapt to sketch-specific patterns and visual ambiguity.

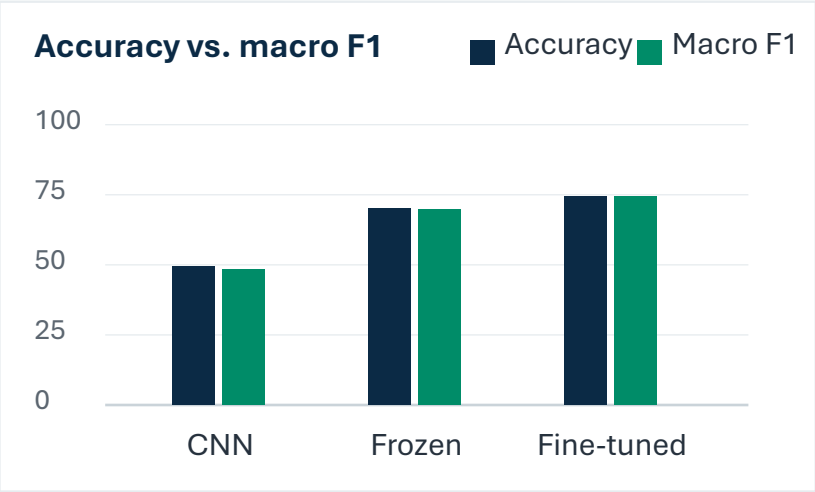
Interpretation

The sequence separates “does it work at all?” from “can pretrained visual knowledge improve recognition?” and “can adapting the feature extractor close the domain gap?”

Fine-tuned EfficientNetB0 wins on accuracy, macro F1, and training time

The custom CNN confirms learnability; transfer learning provides the step change; fine-tuning delivers the best balanced performance.

Model	Accuracy	Macro F1	Training time
Custom CNN	49.41%	48.25%	170.95 min
EfficientNetB0 Frozen	70.12%	69.81%	70.12 min
EfficientNetB0 Fine-Tuned	74.27%	74.23%	45.97 min



+24.86 pp

accuracy improvement over Custom CNN

+25.98 pp

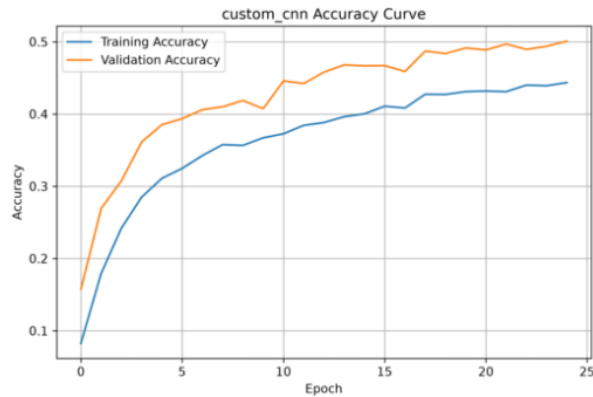
macro F1 improvement over Custom CNN

Best model: EfficientNetB0 Fine-Tuned

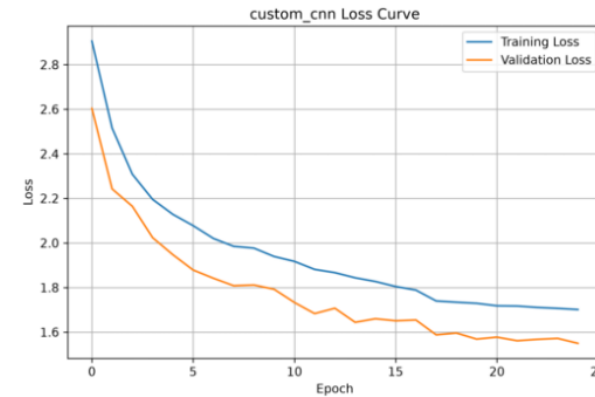
Strongest balanced performance, shortest final training time.

Learning curves show why pretrained features matter

Curated training diagnostics emphasize the baseline learning pattern and the observed transfer-learning performance lift.



Custom CNN accuracy curve



Custom CNN loss curve

Reading the behaviour

- Custom CNN learned basic sketch features, but plateaued at limited generalization.
- EfficientNetB0 started from stronger visual features, improving validation performance substantially.
- Fine-tuning further adapted deeper layers to sketch-specific patterns and delivered the best macro F1.

Visually similar UI elements remain the core challenge

Confusion matrices and misclassified examples point to ambiguity between sketch classes that share geometry, scale, or interaction semantics.

Difficult or confusing pairs

text_field ↔ text_area	similar rectangular text affordances
button ↔ chip	small rounded or pill-like components
dropdown_menu ↔ menu	list-like structures with overlapping cues
switch_enabled ↔ switch_disabled	state differs by subtle stroke placement
alert ↔ card	rectangular containers with internal details

Strong classes in the fine-tuned model

floating_action_button	image
slider	label
grid_list	data_table
radio_button_unchecked	

Interpretation for future work The next improvement step is not only a stronger backbone: explainability and detection-level context can reveal whether errors come from shape ambiguity, missing scale cues, or absent layout context.

Transfer learning turned sketches into measurable UI predictions

Three decisions define the result: structure the sketch data, benchmark from scratch, then adapt pretrained visual features to the UI-sketch domain.

01 Problem

Hand-drawn UI sketches are useful for design thinking, but they are unstructured image data.

02 Approach

We compare a custom CNN baseline, EfficientNetB0 with frozen features, and EfficientNetB0 fine-tuning.

03 Outcome

Fine-tuned EfficientNetB0 is the best model: 74.27% accuracy and 74.23% macro F1.

Executive message

Transfer learning clearly outperformed the custom CNN baseline; fine-tuning improved the model further while reducing training time in the final run.

74.23%

best macro F1-score

Fine-tuned EfficientNetB0

Conclusion

Transfer learning is effective for UI sketch element classification.

Fine-tuned EfficientNetB0 achieved the strongest performance and shows that CNN-based models can classify isolated hand-drawn UI components—while visually similar elements remain challenging.

Limitations

- Visually similar UI elements remain ambiguous
- Hand-drawn variation creates class uncertainty
- Current task classifies isolated elements, not complete layouts

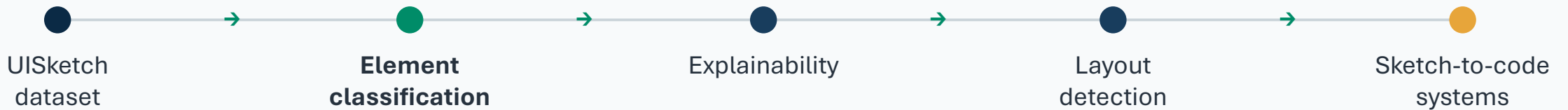
Future work

- Grad-CAM explainability
- More robust augmentation
- MobileNetV2, ResNet50, or Vision Transformer
- Object detection for complete wireframe understanding

Related Work: From UI Sketches to Intelligent Interfaces

A focused step between dataset-driven sketch recognition and broader intelligent-interface systems.

Research evolution



Foundations used directly

1. UISketch Dataset

Large-scale UI element sketches; direct basis for our 21-class task.

4. EfficientNet

Transfer-learning backbone; EfficientNetB0 balances accuracy and cost.

Long-term research context

2. pix2code

Transforms GUI screenshots into code; relevant beyond classification.

3. Sketch2Code

Converts paper mockups into website structures; validates practical sketch understanding.

5. Grad-CAM

Future explainability: highlight sketch regions influencing predictions.

Main message

Classify isolated UI sketch elements reliably and evaluate behavior scientifically before moving to full UI code generation.

References and Paper Bridge

Bridge the oral presentation to the final IEEE-style paper: cited foundations, methodology, results, and discussion.

References

- [1] V. P. Sermuga Pandian, S. Suleri, and M. Jarke, “UISketch: A Large-Scale Dataset of UI Element Sketches,” CHI EA, 2021.
- [2] T. Beltramelli, “pix2code: Generating Code from a Graphical User Interface Screenshot,” 2017.
- [3] A. Robinson, “Sketch2code: Generating a Website from a Paper Mockup,” 2019.
- [4] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” ICML, 2019.
- [5] O. Russakovsky et al., “ImageNet Large Scale Visual Recognition Challenge,” IJCV, 2015.
- [6] R. R. Selvaraju et al., “Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization,” ICCV, 2017.
- [7] K. He et al., “Deep Residual Learning for Image Recognition,” CVPR, 2016.
- [8] A. G. Howard et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” 2017.

Thank you - Questions?

Anjali Jitendrabhai Barvaliya · Chitroda Vishal

IEEE paper structure

Introduction: motivation and problem

Related Work: UISketch; pix2code; Sketch2Code; EfficientNet

Methodology: preprocessing; CNN; transfer learning; fine-tuning

Experiments: dataset split; metrics; training setup

Results: model comparison; confusion matrix; error analysis

Discussion: limitations and future work

Conclusion: final project takeaway